PRÁCTICA 4: MALLAS DEFORMABLES Y ROTURAS

Antonio Jesús Alcaraz Sánchez

Simulación

Grado de Ingeniería Multimedia – ETSE UV

Antonio Jesús Alcaraz Sánchez
ETSE-UV

Índice

Objetivos de la práctica 2

La malla deformable 2

Topología de la malla deformable 3

Muelles 4

Cuestiones planteadas 5

Cuestiones adicionales 6

Antonio Jesús Alcaraz Sánchez
ETSE-UV

Objetivos de la práctica

A lo largo de esta práctica se han implementado y probado diferentes estructuras de datos que pretenden simular mallas deformables con la posibilidad de efectos como la amortiguación, la rotura o la indestructibilidad.

La malla deformable

La base de dichas estructuras es una malla (clase "DeformableSurface") formada por un conjunto de nodos partícula: puntos en un espacio tridimensional con posición, velocidad, masa y cálculo de fuerzas, que en este caso cuenta con características extra:

- Un parámetro condicional **"clamped"** define en cada partícula si contará con movimiento o no, permitiendo establecer los contornos o bordes de la malla que sean estáticos y limiten el movimiento de la tela en la vista en cualquier caso (durante la simulación, existen fuerzas internas que pueden arrugar la tela y fuerzas externas como la gravedad que pueden hacer caer todos los puntos y alejar la malla). Dicha característica se tiene en cuenta al implementar el método creador de nodos:
- Otro parámetro **"_isUnbreakable"** que indicará si los muelles que unen los nodos son destructibles o no.
- Esta condición es evaluada a la hora de crear dichos muelles, al menos los encargados de unir los nodos de la estructura que tienen su origen una vez ejecutado el método

```
/**************
Este método debe dar valores al vector de nodos ('_nodes')
en función del tamaño que tenga la malla y de las propiedades
de ésta (número de nodos, masa de los mismos, etc.).
**************/
void createNodes(float surfacePosZ, float nodeMass)
{
  for(int i = 0; i < _numNodesX; ++i)
    for(int j = 0; j < _numNodesY; ++j)
    {
      PVector s = new PVector(
        (i*_lengthX/_numNodesX) - _lengthX/2,
        (j*_lengthY/_numNodesY) - _lengthY/2,
        surfacePosZ);

      _nodes[i][j] = new Particle(
        s,
        new PVector(),
        nodeMass,
        ((i==0) || (j==0) || (i == _numNodesX -1) || (j == _numNodesY-1)));

    }
}
```

```
/**************
Este método debe añadir muelles a la lista de muelles
de la malla ('_springsSurfaces') en función de la disposición
deseada para éstos dentro de la malla, y de los parámetros
de los muelles (Ke, Kd, etc.).
**************/
void createSurfaceSprings(float Ke, float Kd, float maxForce, float breakLengthFactor)
{
  if(_isUnbreakable) {maxForce = 0; breakLengthFactor = 0;}
  SpringCreatorUtility scu = new SpringCreatorUtility(_springLayout, Ke, Kd, maxForce, breakLengthFactor, this);
  for (int i = 0; i< _numNodesX; i++)
     for (int j = 0; j< _numNodesY; j++)
       scu.doStuff(i,j);
}
```

"createSurfaceSprings()". Puede parecer un método bastante escueto, pero de forma directa e indirecta incluye toda la funcionalidad:

- En caso de ser la estructura indestructible, la fuerza y longitud máxima que pueden aguantar los muelles se indicará como 0. Posteriormente en la actualización del estado de muelles se interpretará como valor nulo.
- **"SpringCreatorUtility"** es una nueva clase que se ha creado en el proyecto para aprovechar las características de herencia y abstracción de Java y llevar a cabo un código algo más escueto y mucho más eficiente. Sus objetos utilizan el **"SpringLayout"**, que indica la topología de la malla actual, para precisamente elegir con cuál de las clases extendidas de un objeto "Creator" inicializar dicho objeto. Esas clases "hijas" implementarán cada una el método abstracto "doStuff()" para cada una de las topologías, habiendo adelantado su selección en un switch mucho antes de invocar cualquier bucle (lo cual evita tener que escribirlo varias veces) y reduciendo la lógica visible en cuanto a la creación de los muelles a un único método "doStuff()".

```
class SpringCreatorUtility
{
  private Creator creator;
  protected float Ke, Kd, maxF, breakLF;
  protected DeformableSurface ds;

  public SpringCreatorUtility(SpringLayout sl, float newKe, float newKd,
          float maxForce, float breakLengthFactor, DeformableSurface newds)
  {
     switch(sl)
     {
       case STRUCTURAL:              creator = new Structural();          break;
       case SHEAR:                   creator = new Shear();               break;
       case STRUCTURAL_AND_SHEAR:    creator = new Structural_Shear();    break;
       case STRUCTURAL_AND_BEND:     creator = new Structural_Bend();     break;
       case STRUCTURAL_AND_SHEAR_AND_BEND:
         creator = new Structural_Shear_Bend();                           break;
     }
      Ke = newKe;
      Kd = newKd;
      maxF = maxForce;
      breakLF = breakLengthFactor;
      ds = newds;
  }

  public void doStuff(int i, int j)
  { creator.doStuff(i,j); }

  private abstract class Creator
  { public abstract void doStuff(int i, int j); }
```

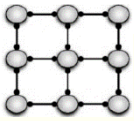
- Como ya hemos mencionado, la forma de simular las interacciones de los nodos de la malla es a través de muelles. Dichos muelles se almacenan en dos estructuras diferentes:
    - *Muelles internos.* Los muelles encargados de simular las tensiones propias de la malla se almacenan en un *ArrayList* rellenado en "**createSurfaceSprings()**".
    - *Muelles de colisión.* Cuando un objeto se acerca, en este caso una esfera, es necesario crear muelles de repulsión que se carguen de la interacción consecuente. Por eso se genera un mapa que incluye o elimina en cada iteración de la simulación los muelles adecuados en base a las condiciones y puertas lógicas descritas en las anotaciones del método encargado: **avoidCollision()**.

```
/**************
Este método evita la colisión entre la esfera y la
malla deformable. Crea muelles de colisión cuando se detecta una.
Estos muelles se almacenarán en el diccionario '_springsCollision'.
Para evitar que se creen muelles duplicados, comprueba si un muelle ya existe.
**************/
void avoidCollision(Ball b, float Ke, float Kd, float maxForce, float breakLengthFactor)
{
  for(Particle [] ap : _nodes)
    for(Particle p : ap)
    {
      boolean colision = (p._s.dist(b._s) < b._r);
      boolean almacenado = _springsCollision.containsKey(p.getId() + "," + b.getId());

      //(colision && almacenado);   Nada
      //(!colision && almacenado);  Borrar
      //(colision && !almacenado);  Guardar
      //(!colision && !almacenado); Nada
      //Solo se hace algo cuando una de las dos condiciones se cumple (XOR)

      //Si solo se cumple una
      if(colision ^ almacenado)
        //Si la que se cumple, es que hay colision (y no almacenada)
        if(colision) _springsCollision.put(p.getId() + "," +b.getId(),
                                           new DampedSpring(b, p, Ke, Kd, true, maxForce,
                                                            breakLengthFactor));
        //Si la que se cumple, es que está almacenada (y no colisiona)
        else _springsCollision.remove(p.getId() + "," +b.getId());
    }
}
```

Topologías de la malla deformable

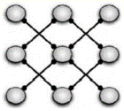
Como se ha mencionado, cada extensión de la clase "Creator" dentro de "SpringCreatorUtility" implementa el método abstracto "doStuff()" para una topología diferente, y este método desarrolla una iteración del bucle de generación de muelles para unos índices i,j de una matriz de nodos. Por lo que mostrar cada una de ellas puede resultar la forma más ilustrativa de explicar cada una de ellas:

```
private class Structural extends Creator
{
  @Override
  public void doStuff (int i, int j)
  {
     if(i < ds._numNodesX-1)        //False pues serán repulsivos y atrayentes
       ds._springsSurface.add(
         new DampedSpring(ds._nodes[i][j], ds._nodes[i+1][j], Ke, Kd, false, maxF, breakLF));

     if(j < ds._numNodesY-1)
       ds._springsSurface.add(
         new DampedSpring(ds._nodes[i][j], ds._nodes[i][j+1], Ke, Kd, false, maxF, breakLF));
  }
}
```



Estructural

Se generan muelles entre un nodo y el que tiene encima y debajo, lo que provoca una unión de todos ellos en cuadrícula, horizontal y verticalmente.

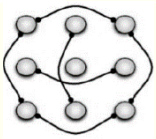


De cizalla

Se generan muelles entre un nodo y el que tiene justo encima formando con los dos siguientes una x, provocando tras el bucle una estructura de uniones en diagonal.

```
private class Shear extends Creator
{
  @Override
  public void doStuff (int i, int j)
  {
     if(i < ds._numNodesX - 1 && j < ds._numNodesY - 1)
     {
       ds._springsSurface.add(
         new DampedSpring(
           ds._nodes[i][j], ds._nodes[i+1][j+1], Ke, Kd, false, maxF, breakLF));
       ds._springsSurface.add(
         new DampedSpring(
           ds._nodes[i][j+1], ds._nodes[i+1][j], Ke, Kd, false, maxF, breakLF));
     }
  }
}
```

```
private class Bend extends Creator
{
  @Override
  public void doStuff (int i, int j)
  {
    new Structural().doStuff(i,j);
    if(i < ds._numNodesX-2)
      ds._springsSurface.add(
        new DampedSpring(ds._nodes[i][j], ds._nodes[i+2][j], Ke, Kd, false, maxF, breakLF*2));

    if(j < ds._numNodesY-2)
      ds._springsSurface.add(
        new DampedSpring(ds._nodes[i][j], ds._nodes[i][j+2], Ke, Kd, false, maxF, breakLF*2));
  }
}
```



Antiarrugas

No se utiliza como un caso individual debido a que no supone una adecuada unión independiente de todos los muelles, aunque sí una herramienta de refuerzo a las anteriores, y es que las 3 estructuras no son excluyentes entre sí y el resto de casos programados a partir de aquí, no son mas que combinaciones:

```
private class Structural_Shear extends Creator
{
  @Override
  public void doStuff(int i, int j)
  {
    new Structural().doStuff(i,j);
    new Shear().doStuff(i,j);
  }
}
```

```
private class Structural_Bend extends Creator
{
  @Override
  public void doStuff(int i, int j)
  {
    new Structural().doStuff(i,j);
    new Bend().doStuff(i,j);
  }
}
```

```
private class Structural_Shear_Bend extends Creator
{
  @Override
  public void doStuff(int i, int j)
  {
    new Structural_Shear().doStuff(i,j);
    new Bend().doStuff(i,j);
  }
}
```

Antonio Jesús Alcaraz Sánchez
ETSE-UV

Muelles

Los muelles planteados para esta simulación cuentan principalmente con dos características destacables, y ambas se encuentra muy presentes en su función de actualización:

```
void update(float simStep)
{
   _F.set(0,0,0);
   if(!_broken)  //l0: long de origen    lr: en reposo    l: actual
   {
     _e = _p2.getPosition().copy().sub(_p1.getPosition());   //vector longi
     _eN = _e.copy().normalize();                            //Dir. ortogor

     float l0 = _l;                                          //longitud de
     _l = _e.mag();                                          //longitud act

     if( _l > _lMax && _lMax > 0  && !_repulsionOnly) {breakIt(); return;}

     float elon = _l - _lr;                                  //Elongación (

     //PVector Fd = PVector.mult(_eN.copy(), (_l - l0) / simStep).mult(_Kd)
     PVector Fd = PVector.sub(_p2._v, _p1._v).mult(_Kd);
     //PVector Fd = new PVector(0,0,0);

     PVector Fm = PVector.mult(_eN, _Ke * elon);             //La fuerza de
     PVector F = PVector.add(Fd, Fm);

     if(_F.mag() > _FMax && _FMax > 0 && !_repulsionOnly) breakIt();
     else _F = F.copy();
   }
}
```

1. Mantendrán una amortiguación en su cálculo de fuerzas. Se han probado 3 implementaciones diferentes: calculándola a través de la velocidad de incremento de longitud del muelle, a través de la substracción de las velocidades de las partículas (en ambos casos orientando en la dirección y sentido adecuado el vector y multiplicándolo por la constante de amortiguación) y con una amortiguación nula (por analizar las bondades en estabilidad que pudiesen ofrecer ambos métodos frente a nada). Si bien se obtiene bastante rigidez, los resultados más adecuados en cuanto a estabilidad y "flacidez" de la malla se encuentra en el segundo caso, descomentado en el código.
2. La gran diferencia entre las fuerzas que actúan de forma interna y externa a la malla es que las primeras forman tensiones (se compensan entre sí) mientras que las segundas provocan un efecto completamente "repulsivo". Es por ello que conviene concebir una booleana con un nombre como "_repulsionOnly" para identificar el caso en que el muelle simboliza una fuerza externa, y evitar fenómenos como la rotura del muelle.

Las fórmulas utilizadas se pueden describir de la siguiente forma:

Fuerza total de un muelle sobre otro	Fuerza de amort. (por veloc. del muelle)	Fuerza amort. (por veloc. de las partículas)
$\overrightarrow{F_t} = \overrightarrow{F_d} + \overrightarrow{F_m}$	$\overrightarrow{F_{d_1}} = \overrightarrow{e_N}\, v_m\, K_d$	$\overrightarrow{F_{d_2}} = (\overrightarrow{v_{p2}} - \overrightarrow{v_{p1}})K_d$

Fuerza del muelle por elongación	Velocidad del muelle	Dirección unitaria del muelle
$\overrightarrow{F_m} = \overrightarrow{e_N}\, K_d\, (l - l_r)$	$v_m = \left(\frac{\partial l}{\partial t} = \lim_{h \to 0} \frac{f(x+h) + f(x)}{h} \rightarrow\right) = \frac{\Delta l}{sim_step}$	$\overrightarrow{e_N} = \frac{\overrightarrow{p_{p2}} - \overrightarrow{p_{p1}}}{l}$

Longitud actual
$l = \lvert\overrightarrow{p_{p2}} - \overrightarrow{p_{p1}}\rvert$

El proceso es sencillo conociendo estas ecuaciones:

1. Comprobar que el muelle aún no está roto.
2. Comprobar si la longitud actual es superior a la máxima (si esta no vale 0 y no es un muelle de repulsión).
3. Si no lo es, calcular la fuerza total.
4. Comprobar si la fuerza total es superior a la máxima (si esta no vale 0 y no es un muelle de repulsión).
5. Si no lo se, la fuerza del muelle es la fuerza total.

Si sí se cumplía alguna de las dos últimas comprobaciones, se rompe el muelle y la fuerza es 0.

```
final PVector G = new PVector(0.0, 0.0, -9.81);   // Accelerat

final float NET_LENGTH_X = 600.0;    // Length of the net in t
final float NET_LENGTH_Y = 400.0;    // Length of the net in t
final float NET_POS_Z = -500.0;   // Position of the net in th
final int NET_NUMBER_OF_NODES_X = 60;   // Number of nodes of
final int NET_NUMBER_OF_NODES_Y = 40;   // Number of nodes of
final float NET_NODE_MASS = 0.1;   // Mass of the nodes of the

//F = Ke * incr(l) - Kd * v = Ke * (l-lr) - Kd * v ; v = veloc
final float NET_KE = 150.0;   // Ellastic constant of the net'
final float NET_KD = 3.0;   // Damping constant of the net's s
final float NET_MAX_FORCE = 500.0;   // Maximum force allowed
final float NET_BREAK_LENGTH_FACTOR = 18.0;   // Maximum dista

boolean NET_IS_UNBREAKABLE = false;   // True if the net canno
SpringLayout NET_SPRING_LAYOUT;   // Current spring layout

final PVector BALL_START_POS = new PVector(0.0, 0.0, -200.0);
PVector BALL_START_VEL = new PVector(0.0, 0.0, -150.0);   // I
final float BALL_MASS = 100.0;   // Mass of the sphere (kg)
final float BALL_RADIUS = 50.0;   // Radius of the sphere (m)

final float COLLISION_KE = 100.0;   // Ellastic constant of th
final float COLLISION_KD = 0.0;   // Damping constant of the c
final float COLLISION_MAX_FORCE = 0.0;   // Maximum force allo
final float COLLISION_BREAK_LENGTH_FACTOR = 18.0;   // Maximum
```

Antonio Jesús Alcaraz Sánchez
ETSE-UV

Cuestiones planteadas

Antes de nada, destacar que todas las preguntas se han realizado utilizando como tarjeta gráfica del procesador Intel i7 de 9 generación, y no en la RTX, además de que durante los primeros ejercicios se ha utilizado el conjunto de parámetros iniciales de la izquierda.

1. ¿Qué diferencias se observan entre los tipos de disposiciones (structural, shear, etc.) de muelles de la malla? ¿Cambia el comportamiento de la simulación? ¿En qué casos?
2. ¿Son todas las disposiciones igual de costosas a nivel de tiempo de computación? ¿Por qué?

Para contestar a estas preguntas, es interesante revisar la parte 1 del vídeo realizado para esta práctica, en el cuál además se compara los dos métodos amortiguadores utilizados. Puesto que en estos vídeos se da gran cantidad de información visual y ciertas anotaciones, me remitiré a dar un conjunto de conclusiones de ellos: https://youtu.be/CbUxILXqz54

- El método de amortiguación por velocidades del muelle ofrece como resultado una tela capaz de estabilizarse, a diferencia de una tela sin amortiguación, pero que cuenta con mayor flacidez, menos interconexión o tirón aparente entre nodos, frente al método de amortiguación por velocidades de las partículas, que genera una estructura mucho más rígida. Incluso este método parece provocar dobleces insalvables en métodos menos estables como el de cizalla.
- Cizalla es muy inestable frente al método estructural, aunque el primero resulta en patrones más vistosos de rotura, pues el método estructural tiende a ocasionar roturas cada vez más unidimensionales conforme la estructura es más rígida.
- Los problemas a la hora de ocasionar roturas pueden solventarse combinándolas, combinando patrones más vistosos y estabilidad.
- El método antiarrugas no es funcional por sí solo; sin embargo, al unir nodos más separados entre sí de lo habitual, cumple la función de su nombre: evitando ciertas arrugas que provocarían sorpasos de la longitud máxima de muelle, creando una estructura rígida difícil de romper. Es de destacar que la programación de los muelles se ha llevado a cabo introduciendo el doble de la longitud de reposo habitual, pues si no, se rompían nada más ser instanciados al sobrepasar por poco menos del doble dicha distancia.
- Tanto aumentar el número de nodos, como introducir el cálculo de la amortiguación (1 fuerza más por muelle) resienten el rendimiento del sistema. Las estructuras que combinan cualquiera de los 3 métodos simples suponen un costo mayor de computo frente a cualquiera de las 2 primeras por separado (no se cuenta antiarrugas por lo explicado).
  A mayor número de muelles o de fuerzas dentro de los muelles, mayor número de cálculos por fotograma.

3. ¿Qué puede ocurrir si la esfera tiene un radio menor que la distancia entre los nodos de la malla? ¿Por qué?

La distancia entre muelles se supone que es 10 píxeles en la configuración base (600/60 o 400/40 de la longitud y el número de nodos). https://youtu.be/CbUxILXqz54?t=38

La colisión depende de la condición de que la distancia al nodo sea menor que el radio de la esfera, por lo que solo existirá en este caso una colisión si la esfera cae a una distancia adecuada entre 2 nodos que cumpla esa condición o si en esta iteración se consigue solapar con un nodo con el que está alineado. Por ejemplo, en la situación inicial, al estar alineada la posición central de la esfera con el nodo central de la malla, con una esfera de radio 8 nunca se detecta colisión con los nodos de alrededor, aunque puede que lo consiga con el nodo central con mucha dificultad. Si movemos la esfera 8 píxeles en cualquier dirección, puede perturbar un poco más la malla, pero en cualquier caso, seguirá atravesándola por la baja incidencia sobre su velocidad.

4. ¿Qué ocurre si la velocidad de la esfera es extremadamente alta cuando la malla es irrompible? ¿y cuándo no es irrompible? ¿Por qué?

En el primer caso, la esfera es demasiado veloz para que los nodos tengan tiempo de verse repelidos por ella y aplicarle una fuerza de impulso contraria. En el segundo, se provocan agujeros inversamente proporcionales a la velocidad, pudiendo incluso no generarlos si la malla es lo suficientemente resistente. En el caso del vídeo, las pruebas se han realizado para una velocidad de 500 y utilizando el amortiguador basado en velocidades de nodo y la topología completa (bend + shear + structural) si perdía algunos muelles shear. https://youtu.be/CbUxILXqz54?t=44

5. ¿Qué ocurre cuando la masa de la esfera es muy grande en relación a la de los nodos de la malla? ¿Por qué?

Depende mucho de la estructura o topología utilizada así como del método de amortiguación. La base del acontecimiento es que la esfera, al tener más masa y por ello más inercia, cede en menor medida ante el empuje de los muelles con la tela. Esto provoca o bien un quiebre más rápido de la topología con un pequeño agujero (un "boquete limpio"), o bien un estiramiento de esta más agudo que desemboque en "latigazos" y mayores dobleces cuando empiece a romperse.
El método de amortiguación por velocidad de los nodos provoca una estructura más rígida que evita estos estiramientos en ciertos casos: la topología estructural da lugar a un efecto en el que se la mesh se rasga horizontalmente, pero no se extiende demasiado. Mientras que la topología de cizalla parecería una alambrada siendo abierta. El otro método permite un mayor estiramiento de la geometría, que además genera olas y la desestabiliza en mayor medida en todos los casos.
En resumen, la esfera cede en menor medida al empuje de la tela y, dependiendo de la rigidez de la estructura, puede tener unos resultados de fisura, agujero limpio o desestabilización con olas. https://youtu.be/CbUxILXqz54?t=66

6. ¿En qué condiciones es la simulación inestable?

Hemos podido varias situaciones de inestabilidad en los casos anteriores: cuando no existe amortiguación o cuando el sistema es de tipo cizalla con un factor de esta bastante bajo, a nivel local o global.
Es evidente por todo lo visto hasta ahora en la asignatura que un SIM_STEP alto debe desestabilizar el sistema, por lo que no pararemos en incluir este caso, ero existen otras dos situaciones que aún no hemos tenido en cuenta:

- Una nula constante elástica en los muelles (Ke) que no mantendría unidos a los nodos ante tensiones externas.
- Cuando la masa de los muelles es demasiado grande, situación que equivale en cierto modo a la anterior al ceder la estructura frente a la fuerza de la gravedad antes que a las tensiones internas. https://youtu.be/CbUxILXqz54?t=92

Cuestiones adicionales

Para contestar estas preguntas, debido a la inclusión de la otra parte opcional en los anteriores apartados de la memoria, del uso de métodos de amortiguación con diferentes resultados y la multitud de ejemplos mostrados, me limitaré a ahondar sobre todo lo visto.

1- ¿Cómo podemos simular una malla que se comporte como una colchoneta elástica?

Las colchonetas son rígidas dentro de lo que cabe, si se trata de una simulación de estado único para un videojuego, puede ser conveniente una mesh con topología estructural-antiarrugas, amortiguador por velocidades de nodos y quizás longitud y fuerza máxima 0 para evitar que se rompa. Si buscamos que se pueda romper, podemos probar:

- Ajustar los parámetros para evitar su rotura por debajo de un determinado peso en kilogramos (humano) a una cierta velocidad de caída libre.
- Utilizar el método por velocidades de muelle, pero con una mayor constante elástica y peso de los nodos.

2- ¿Cómo podemos simular una malla que se comporte como una manta?

En este caso se trata de una malla mucho menos rígida que en el caso anterior. Sus extremos, además, no tendrían por qué estar sujetos. En este caso si es conveniente el método por velocidades de muelle. Podemos introducir un cierto peso y constante de amortiguación para evitar que la malla genere olas y utilizar una topología completa (estructural + antiarrugas + cizalla) para darle profundidad a la geometría y mantener la manta resistente.

3- ¿Cómo podemos simular una malla que se comporte como una alambrada metálica de las que se utilizan para vallar terrenos?

Quizás el mejor ejemplo de esto entre los vistos pueda ser el caso de la estructura en cizalla con amortiguador de velocidades de nodos y una masa proporcionalmente grande, es decir, una masa de nodos pequeña, aunque podría ser conveniente aumentar la constante de amortiguación y reducir la elástica para dotar a los muelles de esa rigidez del metal.

4- ¿Cómo podemos simular una malla que se comporte como una plancha rígida de metal?

Hemos podido ver cómo al aumentar la masa, la estructura se rompía por ceder los muelles periféricos a la gravedad frente a la tensión interna. Si aumentamos la tensión interna a la par que la masa; es decir, la constante elástica, evitaríamos estar rotura y mantendríamos la rigidez. Cualquier topología con estas dos componentes y amortiguación por velocidades de nodo sería rígida para los objetos que colisionen y mantendría el aspecto de una fina plancha de mental que cede levemente a la gravedad, pero se mantiene estable.